

Food Computing Lab Docs

Food Fraud Lifecycle Ontology (FFLO) Knowledge-Graph Method

This document describes the extraction and validation method used to build the food-safety knowledge graph from FSSAI regulations, including the system diagram, worked examples, and evaluation measures. The implementation lives in

`src/extraction/`, `src/validation/`, and `src/kg/`

1. Overview

We extract structured `(subject, predicate, object)` triplets from the FSSAI regulatory corpus in two parallel runs:

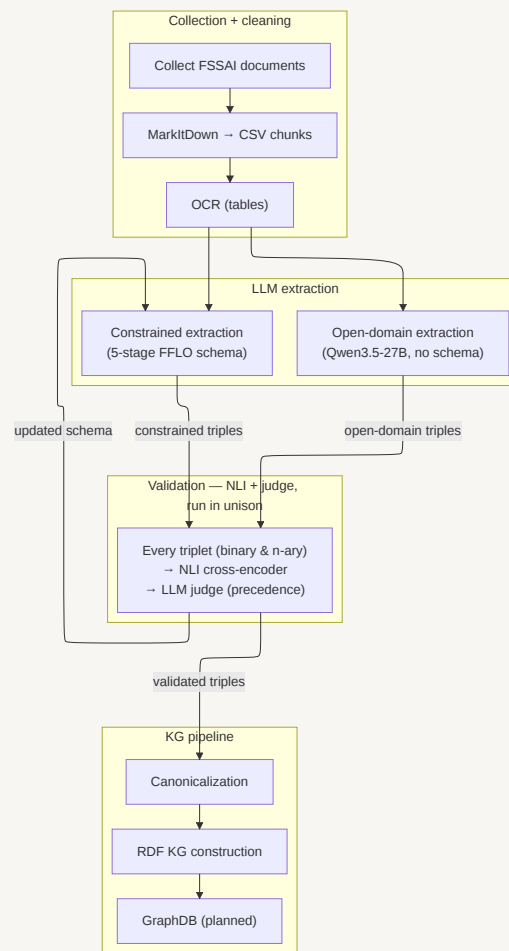
- a **constrained** run, guided by the FFLO ontology schema (5 stages), and
- an **open-domain** run with no schema (Qwen3.5-27B), used to discover what the schema misses.

Each extracted fact is either a **binary relation** (a single subject–predicate–object assertion) or an **n-ary event** (a fact with several linked attributes, reified as multiple triplets sharing one synthetic event id). Every triplet — binary or n-ary, is then validated against its source evidence by a combined **NLI + LLM-judge** layer that runs **both** checks in unison (the judge taking precedence). Validated triples are merged, canonicalised, and exported as a knowledge graph; the open-domain run feeds its discovered gaps back into the schema for the constrained run.

Qwen constrained extraction produced **15,501 triplets** from **1,045 chunks** (5,420 of them n-ary event members); validation yields **13,355 verified triplets**. The open-domain run produced **12,434** free-form triplets, of which **5,882** were judged entailed and drove the schema updates.

This constrained/open-domain split, canonicalised afterward, follows the same general pattern as the Extract–Define–Canonicalize framework for LLM-based KG construction [3], applied here on top of the FKG.in Indian-food ontology [1] (cf. FoodKG [2] as a non-regional precedent for food-domain KG construction).

2. System diagram



3. Extraction

The model is prompted with the FFLO ontology (entity types, relations, and their domain/range constraints) and extracts triplets using **only** those names.

Three behaviours follow directly from the prompt:

1. **Limit decomposition** — a numeric limit is decomposed into a fixed three-relation pattern rather than a single bare relation:
`PermissibleLimit ->forProperty-> Property ,`
`PermissibleLimit ->hasValue-> Measurement ,`
`PermissibleLimit ->inFood-> Food .`
2. **Event reification** — n-ary facts (an event with multiple attributes, e.g. an adulterant *and* a detection method *and* a location) are decomposed into several triplets sharing one synthetic `event_id` .

3. **UNMAPPED** — a fact that cannot be expressed with the current schema is emitted with predicate **UNMAPPED** (and a **mismatch_note**) rather than being forced into an existing relation.

Two extraction modes are used:

Mode	Prompt	Model	Purpose
Constrained (schema-guided)	<code>prompts/triplet_extraction.txt</code>	Qwen	production triplets
Open-domain (unconstrained)	<code>prompts/unconstrained_extraction.txt</code>	Qwen3.5-27B	discover schema gaps

Open-domain (unconstrained) run

The open-domain run extracts from the same chunks with **no schema**: the model freely names entity types and relations from the observed text (e.g. `hasMinSaponificationValue`, `includesSubcategory`). It produced **12,434** free-form triplets. The free-form output is then post-processed in a *flag* pass: entity types and relation names are normalised (string heuristics plus embedding clustering), every triplet is checked by an NLI cross-encoder — with negation handled by rephrasing the hypothesis as “It is NOT the case that ...” and the normalised vocabulary is compared against the FFLO schema to separate covered concepts from candidate new ones. Finally a combined NLI + LLM-judge pass (`nli_llm_judge.py`: chunk-level and span-level NLI scores plus a Qwen judge, with the judge taking precedence; agreed → accepted, disagreed → contested) keeps only the entailed relations: **5,882** of the 12,434 free-form triplets.

The purpose of this run is *coverage* rather than *production*: the entailed free-form triplets surface relation and type concepts the schema did not name. Those gaps become the **updated schema** — 11 additions (e.g. `ObligationType`, `RegulatoryOfficer`, `Microorganism`, `SamplingPlan`, `RegulatoryActionType`, and relations `hasObligation`, `hasScientificName`, `governsSampling`, `isEmpoweredTo`, `actsOnBehalfOf`) — which grows the ontology from the old schema to the new one (89 entity types / 72 relations) and is fed back into the constrained extraction.

Open-domain relation canonicalisation

Because the open-domain run names relations freely, its free-form predicates (e.g. `hasMinSaponificationValue`, `hasMaxLevel`, `includesSubcategory`) must be canonicalised into FFLO relation names before they can drive schema updates. This is a separate, **LLM-grounded** step (`src/validation/unconstrained/llm_canonicalise.py`), distinct from the row-level dedup and entity-ID minting applied to the

constrained run. It reads the similarity-suggested clusters and, for each, pulls grounding examples (subject/object type pairs plus evidence spans) and asks Qwen to **verify each cluster before naming it** — splitting clusters that mix distinct concepts, rejecting out-of-scope/bureaucratic relations, and flagging ambiguous ones for review — with an antonym guard (`include / exclude` , `max / min` , ...) and a confidence threshold. Output is `canonical_map.json` (auto-applied merges) plus `canonical_map_needs_review.json` and `canonical_map_out_of_scope.json` for the rest.

4. Arity: binary vs. n-ary

Facts differ in **arity**:

- **Binary relations** are a single subject–predicate–object assertion (e.g. *Clotted cream* → *belongsToCategory* → *Clotted cream (plain)*).
- **N-ary events** bundle several linked attributes — an adulterant *and* a detection method *and* a location — reified as multiple triplets sharing one synthetic `event_id`.

The arity distinction matters for *why* two validators are used together (the rationale below), but it is **not a hard routing rule**: detecting arity from the extracted structure is a **semantic judgment, not a structural one** — the triplets carry no reliable arity label. Routing by arity would therefore require either manual labelling or an LLM pass built solely to classify arity, and such a pass costs about the same as running the judge directly, which defeats the purpose. So in practice **every triplet — binary and n-ary alike — is passed through both the NLI cross-encoder and the LLM judge**, run in unison with the judge taking precedence (§5).

Why n-ary relations need the judge, not NLI alone

NLI cross-encoders are fine-tuned on **binary premise–hypothesis pairs** (a single premise sentence and a single hypothesis). A multi-argument relation cannot be passed to them directly; there are only two workarounds, and both degrade the check:

1. **Decompose into separate binary checks** — each argument slot becomes its own `(premise, hypothesis)` pair. This *loses the joint constraint* that all arguments must be true *together*; a fact can pass even when only one slot is supported.
2. **Collapse into one compound hypothesis sentence** — all arguments are concatenated into a single hypothesis. This is *out-of-distribution* for

these models, which were not trained on multi-clause conjunctions, and degrades their reliability.

The LLM judge does not have these limitations: it can read the **full evidence span** and check **each argument slot individually and jointly**, which is exactly what an n-ary structure requires. That is why the judge — not the NLI alone — is the authoritative validator for n-ary content.

5. Validation (NLI + LLM-judge, in unison over both runs)

This dual-validator design — a fast entailment model paired with an LLM judge that takes precedence on disagreement — is consistent with recent automatic KG-validation frameworks that combine entailment scoring with LLM judges [4, 5], and with curator-style LLM validation of triple insertions against a target schema [8].

Every triplet — from the constrained run and the open-domain run alike, binary and n-ary — is passed through **both** the NLI cross-encoder and the LLM judge, run in unison. The NLI cross-encoder provides a fast entailment signal by verbalising the triplet into a single premise–hypothesis pair; the LLM judge provides the authoritative verdict, reading the full evidence span and checking each argument slot individually and jointly (plus schema-level corrections). When the two disagree, the **judge takes precedence**; agreement is recorded as `agreed_all`, `agreed_partial`, or `contested`.

Synthetic IDs and verbalisation

NLI models need a single natural-language hypothesis per triplet, but many entities are **synthetic** — pipeline-generated names that never appear verbatim in the text (`ev_dietary_fibre_claim_0001`, `DeclarePotassium`, `Measurement_50_mg_kg`). Before inference, the pipeline decides which entities are synthetic (a name that is not a substring of its evidence span) and assigns each a deterministic hash ID — `sha256(snippet · name · type)[:12]` — so the same synthetic node resolves consistently across all triplets in a chunk (`src/validation/NLI_model/synthetic_ids.py`). Resolution order: CSV `subject_id` / `object_id` columns first, then a gold annotation file, then the heuristic. Each synthetic ID is given a descriptive label from its entity type (`type_phrase`, e.g. “the permissible limit”) and the triplet is verbalised into a sentence using a relation template (`verbalise.py`; e.g. `belongsToCategory` → “{subj} belongs to the food category {obj}”). The cross-encoder then scores that hypothesis against the evidence span (or the full chunk) as the premise.

Binary example

```
Subject   : Clotted cream
Predicate : fflo:belongsToCategory
Object    : Clotted cream (plain)
```

Verbalised hypothesis: *“Clotted cream belongs to the food category Clotted cream (plain).”*

The NLI cross-encoder returns **entailed** (confidence 0.78). The same triplet is also reviewed by the judge, whose verdict takes precedence in the final callout.

N-ary example

The claim “a dietary-fibre *source* claim requires at least 3 g/100 g” is a single n-ary fact, reified as one event `ev_dietary_fibre_claim_0001` with three linked triplets. Each slot is verbalised for NLI, but the authoritative check is the judge’s joint review, which reads the whole evidence span and scores each slot (and the event as a whole):

subject	predicate	object	judge verdict	confidence
<code>ev_dietary_fibre_claim_0001</code>	<code>fflo:forProperty</code>	<code>Dietary Fibre</code>	partially_entailed	0.85
<code>ev_dietary_fibre_claim_0001</code>	<code>fflo:hasValue</code>	<code>3g/100g</code>	partially_entailed	0.85
<code>ev_dietary_fibre_claim_0001</code>	<code>fflo:inFood</code>	<code>food products</code>	partially_entailed	0.85

The judge’s rationale is slot-specific (e.g. *“Text states a minimum threshold for a claim, not a fixed permissible limit value”*).

The judge’s four branches

Branch	Trigger	Verdicts
<code>zero_triplet</code>	chunk produced no triplets	<code>correct_zero</code> , <code>missed_relation</code>
<code>schema_mismatch</code>	relation outside schema	<code>needs_schema_extension</code> , <code>maps_to_existing</code> , <code>unsupported</code>
<code>schema_invalid</code>	domain/range/type failure	<code>entailed_wrong_relation</code> , <code>entailed_wrong_types</code> , <code>entailed_direction_reversed</code> , <code>not_entailed</code>
<code>schema_valid</code>	passed schema checks	<code>entailed</code> , <code>partially_entailed</code> , <code>not_entailed</code>

6. Canonicalisation

Dedup — three-phase row-level canonicalisation

Before entities are assigned IDs, the verified set is deduplicated (`src/extraction/canonicalise_kg.py`). Chunk overlap means the same fact and the same entity are often extracted more than once with small surface-form variations, so the 13,355 verified triplets are reduced in three phases:

1. **Surface-form merge** — merge duplicate entity mentions via fuzzy matching and a graph-neighbour check, guarded against chemical-type collisions (596 fuzzy merges; 278 chemical and 2,864 neighbour cases blocked).
2. **Type-conflict resolution** — resolve genuine type conflicts versus mistyped noise per entity (8,037 resolved; 1,047 genuine conflicts left for review).
3. **Row dedup** — drop near-identical triples via exact (subject, predicate, object) matching plus a Jaccard-based evidence-span pass, taking **13,355** → **11,542** rows (1,813 dropped).

Entity-ID minting

After dedup, entities are canonicalised from raw text spans into shared graph IDs (`fkkg:<TYPE>_NNNNN`) via type-blocking, normalisation, and char n-gram TF-IDF similarity clustering (≥ 0.90 auto-merge, 0.75–0.90 flagged for review). Event-reification IDs (`ev_*`) pass through unclustered as unique per-event identifiers.

An audit of the output found 70 cases where identical entity strings had been split across multiple IDs — caused by inconsistent type labels across mentions (42 cases) and incomplete merge propagation (28 cases), affecting 430 of 13,355 rows. These were fixed with a deterministic remap (keep the most-used ID per cluster), verified via row-count integrity, zero remaining splits, and exact ID-count reconciliation.

A broader near-duplicate scan (cross-type, similarity ≥ 0.70) initially returned 6,151 pairs, but most were false positives from event-IDs, numeric/measurement strings, and values differing only by an embedded number (e.g. different years or thresholds); filtering these down left 2,571 genuine candidates. Of these, only pairs with zero semantic word-level difference (plurals/stopwords only) were auto-merged (92 pairs) — high similarity alone was found unreliable even at ≥ 0.90 (e.g. “Schedule I” vs “II”, “Ethyl” vs “Methyl” falsely scored as near-duplicates). The remainder was left for manual/LLM review rather than auto-merged.

The canonicalisation script (`src/kg/canonicalize.py`) was subsequently rewritten to bake in these findings: similarity-based auto-merging is disabled entirely (only exact-normalised-string matches merge automatically), a new cross-type exact-match pass fixes the type-inconsistency root cause, and guards

were added to block merges across conflicting numeric values or mismatched compound qualifiers.

The final run over the deduplicated verified set assigns **8,003** canonical IDs (1,025 cross-type exact-match merges; 1,811 borderline pairs flagged for review), populating `subject_id` / `object_id`. In the RDF, these map to `urn:fflo:resource:<TYPE>_<NNNN>` instance IRIs (7,996 distinct entities).

7. Evaluation measures

The measures below verify the two runs separately: §7.1–7.4 report the **constrained run** (judge verdicts, support rate, calibration — **15,524** triplets judged, 15,516 successful plus 8 judge errors); §7.5 reports the **open-domain run** (NLI + judge agreement).

7.1 Overall pipeline yield

Stage	Count
Extracted (initial)	15,501
of which event-linked (n-ary)	5,420
Verified (post-judge)	13,355
kept unchanged	11,476
corrected (predicate/type/direction fixed)	1,879
dropped	2,146

Reconciliation: 11,476 kept + 1,879 corrected = 13,355 verified; 15,501 – 13,355 = 2,146 dropped. (The verified CSV holds 13,355 rows but 13,335 distinct keys — 20 rows are exact duplicates of another verified row.)

7.2 Judge verdict distribution

Verdict	Count
<code>entailed</code>	4,930
<code>partially_entailed</code>	2,151
<code>not_entailed</code>	2,507
<code>entailed_wrong_relation</code>	2,315
<code>entailed_wrong_types</code>	3,014
<code>entailed_direction_reversed</code>	552
<code>correct_zero</code>	47
<code>judge_error</code>	8

The table sums to **15,524**: 15,516 successful verdicts plus the 8 `judge_error` rows.

7.3 Judge support-rate by relation type

For each predicate, the support rate is the fraction of *supported* verdicts (`entailed` , `partially_entailed` , `entailed_wrong_relation` , `entailed_wrong_types` , `entailed_direction_reversed`) over all non-error verdicts.

predicate	judged	supported	not supported	support rate
<code>fkg:hasIngredient</code>	1,908	1,525	383	79.9%
<code>fflo:hasFunction</code>	1,851	1,679	172	90.7%
<code>fflo:hasValue</code>	1,752	1,385	367	79.1%
<code>fflo:forProperty</code>	1,690	1,489	201	88.1%
<code>fflo:inFood</code>	1,648	1,202	446	72.9%
<code>fflo:hasDefinition</code>	1,072	1,016	56	94.8%
<code>fflo:hasObligation</code>	1,035	884	151	85.4%
<code>fflo:hasSubcategory</code>	765	689	76	90.1%
<code>fflo:appliesTo</code>	668	607	61	90.9%
<code>fflo:hasScientificName</code>	329	178	151	54.1%
<code>fflo:isEmpoweredTo</code>	314	272	42	86.6%
<code>fflo:hasPermissibleLimit</code>	314	230	84	73.2%
<code>fflo:belongsToCategory</code>	302	243	59	80.5%
<code>fflo:compliesWith</code>	258	250	8	96.9%
<code>fflo:hasAdulterant</code>	144	41	103	28.5%
<code>fflo:hasAdulterationMethod</code>	72	25	47	34.7%

Low-support relations (`hasAdulterant` 28.5%, `hasAdulterationMethod` 34.7%, `hasScientificName` 54.1%) are the ones most often extracted from hedged or procedural text and are the clearest candidates for prompt tightening.

7.4 Confidence calibration

The judge reports a self-reported `judge_confidence` per verdict. This run did not record token-level log-probabilities, so calibration is limited to the self-reported score.

verdict	n	mean confidence
<code>entailed</code>	4,930	0.964

verdict	n	mean confidence
<code>not_entailed</code>	2,507	0.962
<code>entailed_direction_reversed</code>	552	0.926
<code>entailed_wrong_types</code>	3,014	0.912
<code>entailed_wrong_relation</code>	2,315	0.899
<code>partially_entailed</code>	2,151	0.847

The judge is uniformly high-confidence across verdicts (0.85–0.96), with `partially_entailed` the one verdict where confidence drops, consistent with it being the hedged/uncertain category. A fuller calibration would require token-log-probability scoring (available via the judge’s `--use-logprobs` mode, not run for this set).

7.5 NLI-judge agreement (open-domain run)

All triplets pass through both validators, with the judge taking precedence. On the open-domain run — where both NLI and the judge run over every triplet — agreement is tracked as `agreed_all` / `agreed_partial` / `contested`. Of the **12,434** free-form triplets, **5,882** were judged entailed (47.3%). Among the entailed subset the two validators **fully agreed** on 2,760 (46.9%) and **partially agreed** on 3,122 (53.1%); the partial cases are where the NLI signal disagreed with the judge and the judge overrode it. The NLI label split underlines why the judge is needed: at chunk level the cross-encoder said *entailment* 4,147 / *contradiction* 1,304 / *neutral* 431, while at span level it said *entailment* 4,495 / *contradiction* 429 / *neutral* 958 — the premise (chunk vs. span) swings the NLI verdict substantially. On the constrained set the judge is the operative layer — NLI was only run as a small cross-encoder pilot (50 triplets) — so the agreement signal comes from the open-domain run.

8. Notes

`subject_id` / `object_id` and the `fkg:` id namespace

Canonicalisation is a separate downstream step, applied only once the full validated (and deduplicated) set is assembled: clustering entity mentions into stable identifiers needs to see the whole dataset at once, since assigning IDs incrementally risks the same entity receiving different IDs across batches. NLI/judge validation never assigned these IDs, it only checks truth against evidence, not identity, so the ID columns were empty until canonicalisation ran.

The canonicaliser emits instance IDs as `fkg:<TYPE>_NNNN` (e.g. `fkg:F00_00001`). Note the `fkg:` prefix here is the **food-KG instance id namespace**, distinct from the `fkg` *ontology* namespace (`https://w3id.org/fkg/ontology#`). When the RDF is built, these are mapped to `urn:flo:resource:<TYPE>_<NNNN>` instance IRIs so they do not collide with ontology classes. If a pre-existing `subject_id / object_id` is already populated (e.g. by the NLI synthetic-id step), the canonicaliser writes its KG ids to separate `subject_canonical_id / object_canonical_id` columns instead of overwriting.
